Finding Ellipses Final Report

Matteo Zampini

August 2026

Code and data available at: `https://github.com/mtzampini/Finding_Ellipses`

1 Introduction

1.1 The Numerical Range and Kippenhahn's Curve

For an $n \times n$ complex matrix A, the *numerical range* (or field of values) is defined as

$$W(A) = \left\{ \frac{x^* A x}{x^* x} : x \in \mathbb{C}^n \setminus \{0\} \right\}.$$

It is a classical fact (the Toeplitz–Hausdorff theorem) that $W(A)$ is always a compact, convex subset of $\mathbb{C}$. When A is normal, $W(A)$ reduces to the convex hull of the eigenvalues of A; in general, however, $W(A)$ can be an arbitrarily complicated convex region, and describing its boundary explicitly is a nontrivial problem.

Kippenhahn's approach [2, 3] to this problem begins with the *Cartesian (Toeplitz) decomposition* of A into Hermitian parts, following the exposition in [1, Ch. 13],

$$A = H_A + iK_A, \qquad H_A = \frac{A + A^*}{2}, \qquad K_A = \frac{A - A^*}{2i},$$

which is unique, since H_A and K_A are both Hermitian. Because every Hermitian matrix is normal, the numerical range of H_A alone is simply the real interval spanned by its eigenvalues. Kippenhahn's key idea is to recover the boundary of $W(A)$ by tracking how this interval evolves as A is rotated by a phase $e^{-i\varphi}$: the rotation mixes H_A and K_A via

$$H_{e^{-i\varphi}A} = \cos(\varphi)\, H_A + \sin(\varphi)\, K_A,$$

so that the maximum eigenvalue of this rotated Hermitian combination, as φ ranges over $[0, 2\pi)$, traces out the family of support lines of $W(A)$, and hence its entire boundary.

This construction is captured algebraically by the homogeneous polynomial

$$L_A(u, v, w) = \det\big[uH_A + vK_A + wI\big], \tag{1}$$

a degree-n polynomial in three variables. Kippenhahn's theorem states that the curve dual to $L_A(u, v, w) = 0$ (in the sense of projective duality between a curve and its tangent lines) is an algebraic curve $C(A) \subset \mathbb{C}$, the *Kippenhahn curve* (or boundary generating curve), whose convex hull is exactly $W(A)$. The polynomial L_A therefore encodes, in purely algebraic form, all the geometric information about the shape of the numerical range: the coefficients of L_A are, up to normalization, unitary invariants of A, since a unitary conjugation $A \mapsto U^*AU$ leaves H_A and K_A (and hence L_A) unchanged up to the induced action on u, v, w.

1.2 Classification for 3×3 Matrices

For $n = 3$, Kippenhahn gave a complete classification of the possible shapes of $C(A)$, according to the factorization type of L_A [3, Thm. 26]:

(1) L_A factors into three linear terms and $C(A)$ degenerates to three points. This is exactly the case in which A is *normal*: $W(A)$ is the convex hull of the three eigenvalues, a (possibly degenerate) triangle.

(2) L_A factors into a linear term and an irreducible quadratic, so $C(A)$ consists of a point together with an ellipse. This corresponds to matrices that are *unitarily reducible* to a 2×2 block plus a 1×1 block: $W(A)$ is the convex hull of a single eigenvalue and an elliptical numerical range generated by the 2×2 block.

(3) L_A is irreducible and $C(A)$ is a quartic curve with a double tangent and a cusp; the boundary of $W(A)$ contains a *flat portion* (a line segment).

(4) L_A is irreducible and $C(A)$ is a sextic curve consisting of an oval together with an interior three-cusped curve; the boundary of $W(A)$ is smooth (no flat portions), the generic case.

These four cases give a natural, algebraically defined, and unitarily invariant taxonomy of 3×3 matrices, and they are precisely the four classes (NORMAL, REDUCIBLE, FLAT, GENERIC) used throughout this work. A well-known example illustrating case (4) is the nilpotent shift matrix J_3 with ones on the superdiagonal, whose numerical range is the disk of radius $1/\sqrt{2}$ centered at the origin [1, Ch. 13].

1.3 Connection to Blaschke Products and Poncelet's Theorem

The case-(3) boundary curves in Kippenhahn's classification connect directly to a much older and celebrated result in projective geometry, Poncelet's closure theorem, via the theory of Blaschke products — the central three-way connection developed at length in [1], from which the present project takes its name. Loosely speaking, a 3×3 matrix with an irreducible quartic Kippenhahn curve

and a flat boundary segment is associated with a degree-2 Blaschke self-map of the disk, and the closure property of the associated Poncelet polygon (a triangle, in the $n = 3$ case) governs the geometric configuration of $W(A)$ relative to the eigenvalues of A. This link — connecting spectral/algebraic properties of a matrix, the geometry of its numerical range, and the dynamics of Blaschke products on the disk — is the broader mathematical context motivating this project, and is one of the reasons the 3×3 classification is such a natural and well-understood testbed: essentially everything about it is already known in closed form.

1.4 Motivation: From 3×3 Ground Truth Toward the Open 4×4 Case

Kippenhahn's classification for $n = 3$ is exhaustive and constructive, but the analogous classification for $n = 4$ (and beyond) is not fully understood: the possible degrees and singularity structures of the Kippenhahn curve proliferate rapidly with n, and no complete taxonomy comparable to the four 3×3 cases is currently available.

This motivates the central question of this project: *can a neural network, trained only on the coefficients of $L_A(u, v, w)$ — a representation that is well-defined and unitarily invariant for any n — learn to recover the geometric structure of $W(A)$ without being given the underlying algebraic geometry explicitly?* If so, the same feature representation and modeling approach could, in principle, be applied to $n = 4$, where ground-truth labels are not available, potentially surfacing empirical structure (e.g. clusters, decision boundaries, or confidence patterns tied to concrete algebraic quantities) that could inform new conjectures about the 4×4 classification.

The 3×3 case, with its complete and rigorous classification, serves as the ideal controlled setting in which to validate this approach: it is the only setting in which we can both train a model with fully reliable labels *and* rigorously check whether the model's learned representation tracks genuine geometric/algebraic invariants (Section 5) rather than incidental statistical artifacts of the training distribution — a distinction that is essential before any extrapolation to the open $n = 4$ problem can be trusted.

2 Problem Setup

2.1 From Theoretical Classes to Computable Certificates

Section 1.2 described Kippenhahn's classification of 3×3 matrices into four cases, defined in terms of the *factorization type* of the polynomial $L_A(u, v, w)$ in (1). While elegant, this characterization is not directly checkable on a finite-precision, numerically generated matrix: factorization type is an exact algebraic property, and floating-point matrices sampled at random will essentially never satisfy it exactly. To build a labeled dataset, each of the four classes must

therefore be paired with a *numerical certificate* — a scalar quantity, computable from A, that degenerates to (or approaches) zero exactly when A belongs to that class, together with a threshold below which the class membership is accepted.

We adopt the following four certificates, matching the four cases of Section 1.2 and implemented in `matrixgen/generator.py`.

Case 1 — Normal

A matrix is normal iff $AA^* = A^*A$. We measure deviation from normality with the relative commutator norm

$$\mathrm{cert}_1(A) = \frac{\|AA^* - A^*A\|_F}{\|A\|_F^2}, \tag{2}$$

accepted when $\mathrm{cert}_1(A) \le \tau_1 = 10^{-12}$ (i.e. zero up to floating-point round-off). Case-1 matrices are generated *constructively* rather than by rejection sampling: $A = Q\,\mathrm{diag}(\lambda_1, \lambda_2, \lambda_3)\,Q^*$ for a Haar-random unitary Q and random (generically distinct) eigenvalues $\lambda_i \in \mathbb{C}$, which is normal by construction, so cert_1 serves purely as a sanity check rather than an acceptance filter.

Case 2 — Reducible

A matrix is (unitarily) reducible to a 2×2 block plus a 1×1 block if it has a common eigenvector shared by A and A^* associated to some eigenvalue λ_0. Case-2 matrices are constructed explicitly: a 2×2 block B with prescribed eigenvalues λ_1, λ_2 and off-diagonal coupling, together with a third eigenvalue λ_0 sampled either inside or outside the elliptical numerical range of B (governed by the boolean flag `inside`, itself sampled with probability $1/2$), are assembled into a block-diagonal matrix and then conjugated by a random unitary U. The certificate verifies that the third column $q = U_{:,3}$ is indeed a joint eigenvector of A and A^*:

$$\mathrm{cert}_2(A) = \frac{\max\big(\|Aq - \lambda_0 q\|,\ \|A^*q - \overline{\lambda_0}\, q\|\big)}{\max\big(\|A\|_F,\ \varepsilon_{\mathrm{mach}}\big)}, \tag{3}$$

accepted when $\mathrm{cert}_2(A) \le \tau_2 = 10^{-12}$. As with Case 1, this is a construction-time sanity check rather than a rejection filter.

Cases 3 and 4 — Flat vs. Smooth Boundary

Unlike Cases 1–2, Cases 3 and 4 cannot be constructed exactly in closed form; instead we rely on the support-line construction of Section 1.1 to *measure* how close the boundary of $W(A)$ comes to containing a flat (line-segment) portion, and accept or reject candidate matrices accordingly.

For $H = H_A$, $K = K_A$ and $\theta \in [0, 2\pi)$, define

$$M(\theta) = \cos(\theta)\, H + \sin(\theta)\, K,$$

and let $\mu_1(\theta) \ge \mu_2(\theta)$ denote the two largest eigenvalues of $M(\theta)$ (a Hermitian matrix, so its eigenvalues are real). The *support gap* at angle θ is $g(\theta) =$

$\mu_1(\theta) - \mu_2(\theta)$; a flat portion of the boundary of $W(A)$ corresponds to an angle at which the maximal eigenvalue of $M(\theta)$ has multiplicity ≥ 2, i.e. $g(\theta) = 0$. We evaluate g on a uniform grid of $n_\theta = 720$ angles and define

$$\text{rel\_gap}(A) = \frac{\min_\theta g(\theta)}{\max_i |\lambda_i(H)| + \varepsilon}, \tag{4}$$

the minimal support gap normalized by the typical scale of H (with $\varepsilon = 10^{-9}$ to avoid division by zero).

- **Case 3 (Flat):** H is constructed with a repeated eigenvalue ($\text{diag}(a, a, b)$ in a random unitary basis, with $|a - b|$ bounded below) and K random Hermitian; the pair (H, K) is accepted as Case 3 if

$$\text{rel\_gap}(A) \leq \tau_3 = 2 \times 10^{-3}.$$

- **Case 4 (Generic):** H and K are both random Hermitian matrices (no repeated-eigenvalue structure imposed); the pair is accepted as Case 4 if

$$\text{rel\_gap}(A) > \tau_4 = 10^{-3},$$

  resampling up to 100 times if the sampled matrix accidentally lands below threshold.

Note that $\tau_3 \neq \tau_4$: the two thresholds are not complementary halves of a single decision boundary, but independently chosen operating points for two separate rejection-sampling procedures, each tuned to keep its own class numerically well-separated from the ambiguous region near $\text{rel\_gap} \approx 0$. This choice is a practical, empirically-tuned modeling decision (Section 6 revisits its implications) rather than a value derived from the theory itself, which only requires $\text{rel\_gap} = 0$ exactly for a true flat boundary.

2.2 Feature Representation

For each generated matrix A, the input to the classifier is not A itself but the coefficient vector of its Kippenhahn polynomial $L_A(u, v, w)$ from (1), computed symbolically once (via `sympy`, see `matrixgen/kippenhahn_symbolic.py`) as a function of the 9 real and 9 imaginary independent entries of H_A and K_A, and then evaluated numerically (`lambdify`) for each sampled matrix. $L_A(u, v, w)$ is homogeneous of degree 3 in (u, v, w), hence has $\binom{3+3-1}{3} = 10$ monomial coefficients; the coefficient of w^3 is identically 1 (since $L_A(0, 0, w) = \det(wI) = w^3$) and is therefore dropped, leaving a **9-dimensional feature vector** per matrix. Before extraction, A is normalized by its Frobenius norm, so the feature vector is invariant to overall scaling of A in addition to being invariant under unitary conjugation $A \mapsto U^*AU$ (Section 1.4) by construction of L_A itself.

This 9-dimensional representation is the key design choice that makes the approach potentially transferable to $n = 4$: the construction $A = H_A + iK_A \mapsto$

$L_A(u, v, w) \mapsto$ (coefficient vector) is defined verbatim for any n, only the dimension of the resulting feature vector changes. For a homogeneous degree-n polynomial in 3 variables, the number of monomials is $\binom{n+2}{2} = \frac{(n+1)(n+2)}{2}$; dropping the trivial w^n coefficient (always 1) leaves

$$d(n) = \frac{(n+1)(n+2)}{2} - 1$$

nontrivial invariant features. This correctly gives $d(3) = 9$ (the case used throughout this work) and $d(4) = 14$ for the 4×4 case, meaning a classifier for $n = 4$ would require only a change of input dimension from 9 to 14 in the architecture of Section 3, with no other structural change to the feature-extraction pipeline.

3 Dataset, Model Architecture and Training Setup

3.1 Dataset Construction

The dataset is generated by `matrixgen/dataset_builder.py` according to the configuration in `matrixgen/dataset_config.py`. For each of the four classes, a base pool of matrices is sampled using the corresponding constructive/rejection procedure of Section 2, with class proportions

$$p_1 = 0.15 \text{ (Normal)}, \quad p_2 = 0.25 \text{ (Reducible)},$$
$$p_3 = 0.10 \text{ (Flat)}, \quad p_4 = 0.50 \text{ (Generic)},$$

out of $n_{\text{core}} = 8000$ base matrices in total (1200, 2000, 800, 4000 matrices respectively).

Train/validation split. This base pool is split into train and validation sets *before* any data augmentation is applied, using a stratified split with 80% of matrices assigned to train and 20% to validation (`train_fraction`= 0.8), stratified on the class label to preserve class proportions in both splits. This ordering — split first, augment only the training portion afterward — is a deliberate design choice: augmenting before splitting would let unitarily-conjugated copies of the same base matrix appear in both train and validation, leaking information between the two sets.

Data augmentation. Only the training split is augmented. For each class c, every base matrix A in that class is replaced by itself together with k_c additional copies $U_i^* A U_i$ for independent Haar-random unitaries U_i, where

$$k_1 = 5, \quad k_2 = 5, \quad k_3 = 8, \quad k_4 = 5.$$

Since the Kippenhahn coefficients (Section 2.2) are unitarily invariant by construction, these augmented copies carry *identical* feature vectors to their base

matrix; augmentation therefore does not add new points in feature space, but is used to compensate the smaller base pool of Case 3 (the least represented class pre-augmentation, 10% of the base pool) with a larger multiplicative factor ($k_3 = 8$ vs. $k_c = 5$ for the other classes), partially rebalancing the final class distribution.

Standardization. After feature extraction, each of the 9 features is standardized (zero mean, unit variance) using statistics computed *only* on the training split; the same mean/std are then applied to the validation split, avoiding any leakage of validation statistics into the preprocessing pipeline.

Resulting class distribution. Table 1 reports the exact class counts obtained with this configuration (verified directly from the saved dataset arrays).

Class	Train	Validation
Normal (1)	5,760	240
Reducible (2)	9,600	400
Flat (3)	5,760	160
Generic (4)	19,200	800
Total	40,320	1,600

Table 1: Class distribution after augmentation (train) and stratified split (validation). The ratio between the largest and smallest training class is $19{,}200/5{,}760 \approx 3.33$.

3.2 Model Architecture

We compare two architectures built from the same base module, a small multilayer perceptron (`MatrixClassifierMLP`):

$$f_\theta(x) = W_3\,\sigma\big(W_2\,\sigma(W_1 x + b_1) + b_2\big) + b_3, \qquad \sigma = \mathrm{ReLU}, \tag{5}$$

with input dimension 9 (Section 2.2), two hidden layers of width 32 and 16, and no activation on the output layer (raw logits, consistent with the use of `CrossEntropyLoss`, which applies `log_softmax` internally). No dropout, batch normalization, or weight decay is used in the base architecture; regularization is provided solely by early stopping (Section 3.3).

Flat classifier. A single instance of (5) with 4 output logits, trained end-to-end to directly predict one of the four classes.

Hierarchical classifier. Three independent instances of (5), each with 2 output logits, trained as binary classifiers on nested subsets of the data, mirroring

the order in which Kippenhahn's classification (Section 1.2) naturally partitions matrices:

- **Node A** (Normal vs. non-Normal): trained on the *full* dataset, label 1 iff class = 1.
- **Node B** (Reducible vs. irreducible): trained only on matrices with true class in $\{2, 3, 4\}$ (i.e. non-normal), label 1 iff class = 2.
- **Node C** (Flat vs. Generic): trained only on matrices with true class in $\{3, 4\}$, label 1 iff class = 3.

At inference time, a new matrix is routed through the cascade A $\rightarrow$ B $\rightarrow$ C: it is assigned class NORMAL if Node A predicts 1; otherwise it is passed to Node B, assigned class REDUCIBLE if Node B predicts 1; otherwise it is passed to Node C, which makes the final FLAT/GENERIC decision. Because each node in the cascade only sees the subset of examples routed to it by the previous node, an error made at Node A can never be corrected downstream — a structural property that plays a central role in the results of Section 4.

Each of the four models (flat + 3 nodes) has its own independently computed class weights (Section 3.3) and is trained separately, with its own early-stopping checkpoint.

3.3 Training Hyperparameters

All four models are trained with the same protocol, differing only in the data and class-weight vector used:

- **Optimizer:** Adam, learning rate 10^{-3}, default β_1, β_2.
- **Batch size:** 32.
- **Loss:** weighted cross-entropy. Writing $n_1, \ldots, n_C$ for the per-class counts in the (node-specific) training set, the weight assigned to class c is

$$w_c = \frac{\sum_{c'} n_{c'}}{C \cdot n_c}, \tag{6}$$

  i.e. inversely proportional to class frequency, computed separately for the flat classifier ($C = 4$) and for each binary node ($C = 2$), using `np.bincount` on the respective training labels to guard against a class being entirely absent from a node's training subset.

- **Early stopping:** training runs for at most 100 epochs, monitoring validation loss after each epoch; the best model (lowest validation loss) is checkpointed, and training halts if validation loss fails to improve by more than `min_delta` = 0.001 for `patience` = 10 consecutive epochs. The checkpointed best-validation-loss weights, not the final-epoch weights, are used for all evaluation in Section 4.

- **Reproducibility:** `numpy` and `torch` random seeds are fixed to 42 prior to dataset generation and prior to each training run.

This protocol was arrived at empirically: an initial run without early stopping, trained for the full 100 epochs, showed clear overfitting (validation loss reaching a minimum around epoch 15–20 before increasing monotonically while training loss continued to decrease), motivating the early-stopping criterion adopted here. This diagnostic process is documented in Section 4.

4 Results

4.1 Training Dynamics and the Need for Early Stopping

An initial training run of the flat classifier for a fixed 100 epochs, without early stopping, revealed clear overfitting: validation loss reached a minimum around epoch 15–20 and then increased monotonically for the remainder of training, while training loss continued to decrease throughout — the classic signature of the model beginning to fit noise specific to the training set rather than generalizable structure. This motivated adopting the early-stopping protocol described in Section 3.3 for all subsequent runs, which reduced (but did not eliminate) the train/validation gap.

Figure 1 shows the loss and accuracy curves for the final flat-classifier run, with training halted by early stopping. Figure 2 shows the corresponding curves for the three binary nodes of the hierarchical classifier.

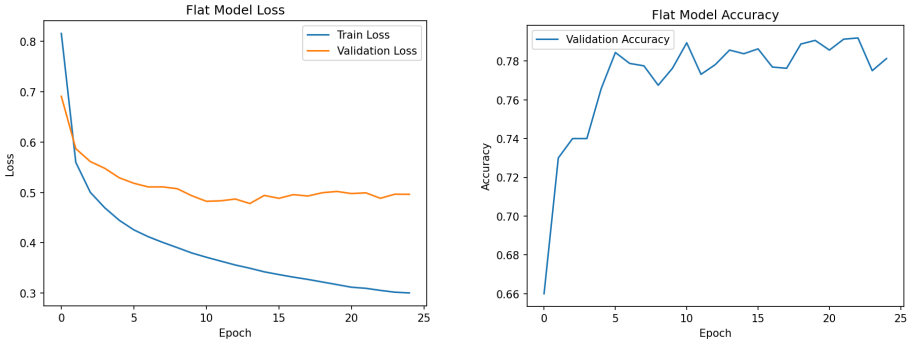

Figure 1: Flat classifier: training/validation loss (left) and validation accuracy (right) per epoch. Early stopping halts training around epoch 25, well before the divergence pattern seen in the earlier unregularized run.

4.2 Flat Classifier

Table 2 reports precision, recall and F1-score per class on the validation set (evaluated using the early-stopping checkpoint, not the final-epoch weights), and Figure 3 shows the corresponding confusion matrix.

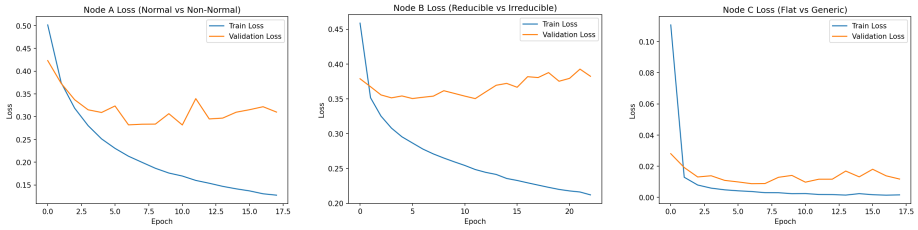

Figure 2: Hierarchical classifier: training/validation loss per epoch for Node A (Normal vs. non-Normal), Node B (Reducible vs. irreducible), and Node C (Flat vs. Generic). Note the different loss scale for Node C (< 0.03 throughout), reflecting the fact that this binary task is comparatively easy once restricted to the Flat/Generic subset.

Class	Precision	Recall	F1-score	Support
Normal	0.621	0.821	0.707	240
Reducible	0.664	0.720	0.691	400
Flat	0.946	0.981	0.963	160
Generic	0.900	0.769	0.829	800
Accuracy			0.786	1600
Macro avg	0.783	0.823	0.798	1600
Weighted avg	0.804	0.786	0.790	1600

Table 2: Flat classifier: per-class validation metrics.

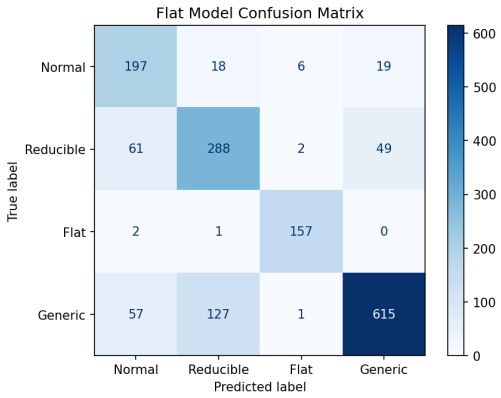

Figure 3: Flat classifier confusion matrix (rows: true class, columns: predicted class).

The Flat class is classified almost perfectly (F1 = 0.963), consistent with the fact that its defining certificate (Eq. 4, small rel_gap) produces a comparatively distinctive coefficient signature. Normal and Reducible are the two weakest classes (F1 0.707 and 0.691), and the confusion matrix shows the dominant error is **not** between Normal and Reducible themselves (only $18 + 61 = 79$ combined misclassifications between them) but between **Reducible and Generic**: 49 true-Reducible matrices predicted as Generic and 127 true-Generic matrices predicted as Reducible, for 176 combined errors — by far the largest confusion pair in the matrix, despite Reducible and Generic occupying non-adjacent branches of Kippenhahn's classification tree (Section 1.2).

4.3 Hierarchical Classifier

Table 3 and Figure 4 report the same metrics for the hierarchical cascade (Section 3.2), obtained by routing every validation matrix through Nodes A→B→C and comparing the final routed prediction against the true 4-class label.

Class	Precision	Recall	F1-score	Support
Normal	0.532	0.925	0.676	240
Reducible	0.675	0.642	0.658	400
Flat	0.944	0.956	0.950	160
Generic	0.916	0.733	0.814	800
Accuracy			0.761	1600
Macro avg	0.767	0.814	0.775	1600
Weighted avg	0.801	0.761	0.768	1600

Table 3: Hierarchical cascade: per-class validation metrics.

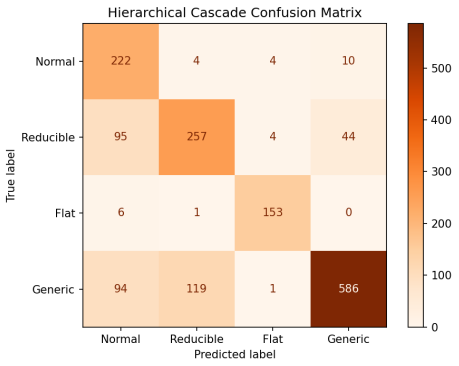

Figure 4: Hierarchical cascade confusion matrix (rows: true class, columns: predicted class).

Node A reaches very high recall on Normal (0.925, up from 0.821 in the flat model), at the cost of aggressively over-predicting NORMAL: 95 true-Reducible and 94 true-Generic matrices are routed to NORMAL by Node A alone (compared to 61 and 57 respectively for the corresponding flat-model errors), and — because the cascade structure of Section 3.2 makes Node A's decision final and unreviewable by downstream nodes — these are errors that no amount of accuracy at Nodes B or C can subsequently correct.

4.4 Flat vs. Hierarchical Comparison

Figure 5 compares per-class F1-scores directly. The flat classifier matches or outperforms the hierarchical cascade on every single class, and by a wide margin on Normal (0.707 vs. 0.676) and Reducible (0.691 vs. 0.658); overall validation accuracy is 78.6% for the flat model against 76.1% for the hierarchical cascade.

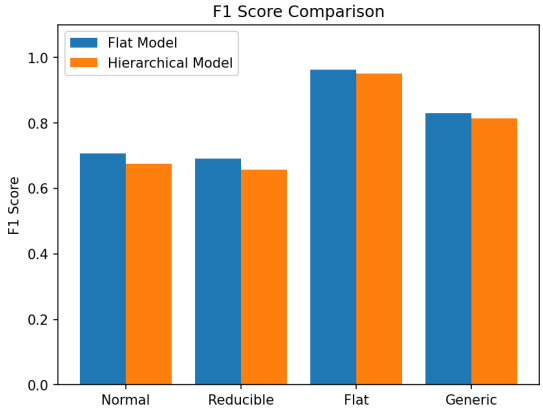

Figure 5: Per-class F1-score, flat classifier vs. hierarchical cascade.

This result runs counter to the initial hypothesis motivating the hierarchical design (Section 3.2) — namely, that decomposing the 4-way decision into a sequence of binary decisions mirroring Kippenhahn's classification tree would let each sub-model specialize and outperform a single 4-way classifier, particularly on the Reducible/Generic confusion identified in Section 4.2. Instead, the cascade structure introduces *unrecoverable error propagation*: Node A's higher recall on Normal is achieved by lowering its decision threshold for that class, which increases the number of non-Normal matrices misrouted into the Normal branch — errors that a flat, jointly-trained 4-way softmax does not make in the same way, since it weighs evidence for all four classes simultaneously rather than sequentially. We take this as empirical evidence that, for this feature representation and dataset, the flat classifier is the better-performing architecture, and use it as a basis for the interpretability and 4×4-oriented analysis of Section 5.

5 Toward $n = 4$: Interpretability and a Confidence–Certificate Probe

The results of Section 4 establish that a small MLP, trained only on the 9 unitarily-invariant coefficients of $L_A(u, v, w)$, can recover the 3×3 Kippenhahn classification with reasonable accuracy (78.6%, and near-perfect on the Flat class). Because the ground truth for $n = 3$ is fully known, this setting doubles as a controlled testbed for a question that matters specifically for extending the approach to $n = 4$, where no such ground truth exists: *is the network's decision driven by genuine geometric structure in the coefficients, or by incidental statistical regularities of the training distribution?* This section reports two preliminary probes aimed at this question. Both should be read as an initial, proof-of-concept methodology — a starting point for validating transferability to $n = 4$ — rather than a conclusive answer; each points to a concrete follow-up described in Section 6.

5.1 Feature Importance

As a first, coarse probe, we inspect the first-layer weight matrix $W_1 \in \mathbb{R}^{32\times 9}$ of the trained flat classifier (Eq. 5). Since the 9 input features are standardized (Section 3.1) prior to training, the columns of W_1 are directly comparable, and we take $\text{imp}_j = \frac{1}{32}\sum_i |W_{1,ij}|$ as a proxy for how strongly feature j contributes to the first hidden representation.

The resulting importances range narrowly, from 0.29 to 0.42 across the 9 features, with no single coefficient or small subset dominating the others by a wide margin. We read this as a mildly encouraging, if inconclusive, sign: the network does not appear to be relying on a single "shortcut" coefficient (which would show up as one or two features with importance far above the rest), but rather integrates information distributed across most of the 9-dimensional invariant representation — consistent with, though not proof of, the network using genuine multi-coefficient geometric structure rather than an easily-identifiable heuristic. A firmer conclusion would require an ablation study (retraining with individual features masked) rather than weight magnitude alone; we list this as a natural next step in Section 6.

5.2 Confidence vs. a Continuous Geometric Certificate

The feature-importance probe above is necessarily specific to the 9-dimensional, $n = 3$ representation and does not directly generalize. A more transferable diagnostic is to compare the network's predictive confidence against a certificate quantity that is well-defined for *any* n — here, the rel_gap certificate of Eq. 4, which underlies the Flat/Generic distinction (Cases 3 and 4) and requires no ground-truth label to compute, only H_A and K_A.

We construct a path of matrices $M(t) = (1 - t)A' + tB'$, $t \in [0, 1]$, where A' and B' are sampled directly from `gen_case3` and `gen_case4` (Section 2) —

i.e. two matrices whose true class membership is certified by construction, isolating a single decision boundary (Flat vs. Generic) rather than mixing multiple boundaries along the path. At each t, we compute rel_gap$(M(t))$ directly from $H_{M(t)}, K_{M(t)}$ and compare it against the flat classifier's confidence $\max_c \text{softmax}(f_\theta(x_{M(t)}))_c$ on the corresponding (scaled) feature vector. Figure 6 shows the result.

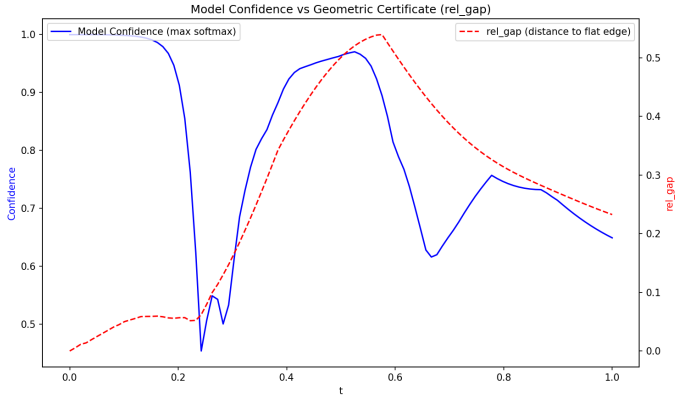

Figure 6: Model confidence (blue, left axis) vs. rel_gap (red dashed, right axis) along a path between a certified Case-3 matrix ($t = 0$) and a certified Case-4 matrix ($t = 1$).

Two regimes are visible. For $t \gtrsim 0.3$, confidence and rel_gap track each other closely: both rise together to a joint peak around $t \approx 0.55$–0.6, then decline together toward $t = 1$ — over roughly 70% of the path, higher geometric distance from the flat-boundary condition coincides with higher model confidence, exactly the qualitative relationship one would hope for if the model's confidence reflects the underlying certificate. Near the very start of the path ($t \lesssim 0.15$), confidence is also correctly near its maximum (≈ 0.99), consistent with $M(0) = A'$ being deep inside the certified Case-3 region (rel_gap ≈ 0).

The one clear departure from this pattern is a sharp, localized confidence dip (down to ≈ 0.45) around $t \approx 0.15$–0.3, occurring while rel_gap itself remains small and only slowly increasing in that same interval — a regime the certificate does not flag as unusual, but where the model's confidence drops sharply nonetheless. We do not have a verified explanation for this localized anomaly; a natural next step (Section 6) is to inspect the individual coefficient trajectory in that interval directly, rather than only the single scalar rel_gap, to check whether some other, unmeasured aspect of the path (e.g. proximity to the Reducible region, which is not certified against on this path) is responsible.

5.3 Summary and Implication for $n = 4$

Taken together, these two probes give a modest but genuine positive signal: the model does not appear to rely on a single dominant feature, and its confidence broadly co-varies with an independently-computed, ground-truth-free geometric quantity over most of a path that isolates a single decision boundary. Both diagnostics — weight-based feature importance and confidence-vs-certificate probing — rely only on ingredients that are defined for general n: the $d(n)$-dimensional coefficient vector of Section 2.2, and certificates such as rel_gap and cert_1 (Eqs. 4, 2) that are computable directly from H_A, K_A without requiring a known classification. This makes them, in principle, directly applicable at $n = 4$ once a classifier is trained there: even without 4×4 ground-truth labels, one could (i) generate 4×4 matrices via analogous constructive/certificate-based procedures for whichever cases *are* understood in closed form, (ii) train a 14-dimensional-input classifier (Section 2.2) on those, and (iii) apply the same confidence-vs-certificate methodology developed here to assess whether the resulting model's confidence tracks genuine algebraic structure — using this 3×3 study as the calibration reference for what a genuine, versus spurious, confidence-certificate relationship looks like.

6 Limitations

A few aspects of this study are worth noting for context. The dataset is split only into training and validation sets (Section 3.1); since validation performance also guided early-stopping and architecture choices, the reported metrics are likely mildly optimistic relative to a held-out test set. Data augmentation is limited to unitary conjugation and does not include a global phase rotation $A \mapsto e^{i\varphi}A$; since H_A, K_A (and hence L_A) do depend on φ, this is a source of variation not explicitly covered during training. The two acceptance thresholds τ_3, τ_4 (Section 2) leave a small unlabeled gap in rel_gap that is resampled away during generation rather than resolved. The feature-importance analysis of Section 5.1 is based on first-layer weight magnitudes only, which is suggestive but not a causal measure; an ablation study would give a firmer answer. Finally, the localized confidence anomaly noted in Section 5.2 remains unexplained and would benefit from a closer look at the individual coefficient trajectories in that region.

7 Conclusion

This work trained neural classifiers to recover Kippenhahn's four-class taxonomy of 3×3 matrices directly from the unitarily-invariant coefficients of $L_A(u, v, w)$, without any explicit geometric or algebraic input beyond this 9-dimensional representation. A flat, jointly-trained 4-way classifier reached 78.6% validation accuracy, correctly separating the Flat class almost perfectly while confusing

Reducible and Generic matrices most often; a hierarchical cascade built to mirror Kippenhahn's decision tree underperformed the flat model on every class, illustrating that decomposing a classification problem along its "natural" mathematical structure does not automatically translate into a better-performing model when errors at an early stage cannot be corrected downstream.

Because the 3×3 case admits a complete, closed-form classification, it served here as a controlled setting in which to ask a question that is unanswerable for $n = 4$ directly: does the trained model's confidence reflect genuine geometric structure, or artifacts of the training distribution? The two preliminary probes of Section 5 — first-layer feature importance and confidence tracked against an independently-computed certificate along a boundary-crossing path — gave a modestly encouraging answer, with no single dominant feature and a broad co-movement between confidence and the geometric certificate over most of the path tested, alongside one unexplained anomaly worth further investigation.

The invariant coefficient representation, the certificate quantities, and the confidence-probing methodology developed here are all defined for general n, and require no 4×4 ground truth to apply. The natural next step is to instantiate this same pipeline at $n = 4$: generate matrices for whichever cases remain constructible or certifiable in closed form, train a 14-dimensional-input classifier, and use the diagnostics validated here as the standard against which to judge whether its learned confidence is trustworthy — turning a classification exercise on a solved problem into a calibrated tool for probing an open one.

References

[1] Ulrich Daepp, Pamela Gorkin, Andrew Shaffer, and Karl Voss. *Finding Ellipses: What Blaschke Products, Poncelet's Theorem, and the Numerical Range Know about Each Other*, volume 34 of *Carus Mathematical Monographs*. American Mathematical Society / Mathematical Association of America, Providence, RI, 2018.

[2] Rudolf Kippenhahn. Über den wertevorrat einer matrix. *Mathematische Nachrichten*, 6:193–228, 1951.

[3] Rudolf Kippenhahn. On the numerical range of a matrix. *Linear and Multilinear Algebra*, 56(1–2):185–225, 2008. Translated from the German by Paul F. Zachlin and Michiel E. Hochstenbach; original: Kippenhahn [2].

Acknowledgments

The author made use of AI (Claude, Anthropic) as a coding assistant throughout development and for assistance in drafting this report.